MYGROKFLOW

OPERATOR PLAYBOOK

Systems that run without you

Write this into your files. Influence how your team works.
Finish one painful workflow in seven days.

mygrokflow.com

Lead generation · Customer service · Documents · Invoicing · Data hygiene

A MyGrokFlow playbook you fill in. Agent OS, loop engineering, AEGIS-style harness rules, and the war-room principal brief — written so you can change them, teach them, and run them.

How to use this

This is one public magnet. Read a section. Write the one-line definition from memory. Do the drill. Then put the files on disk.

Chat still has a job: calories in a banana, hotel distance, a screenshot question. Anything that should leave your plate belongs to an agent.

Three layers in this book

Layer	Job	When you feel it
Agent OS	Context, skills, tools, memory as owned files	Every session
Loop engineering	Perceive → Reason → Act → Observe until done	Every job
Harness engineering	The scaffolding around that loop	When the same job keeps drifting

What this book is not

- Not a transcript of someone else’s episode.
- Not a prompt pack you paste into ChatGPT Projects.
- Not a live trading system and not a replacement for your CRM.
- Not permission to send, scrape at scale, or skip the law in your country.

This is MyGrokFlow’s operator install. Change the files. Do not keep it as a PDF you never open.

Part I · Agent OS

1. Chat vs agent

Chat is question to answer. You still do the work. An agent is goal to result. It plans, acts, and hands back a finished artifact.

	Chat	Agent
Input	A question	A goal with a done-state
Output	An answer	A file, a send behind a gate, a ticket closed
Who works	You	The model inside a harness
Best for	Lookups, small decisions	Multi-step work that should leave your plate

Drill. Take one task from yesterday. Write it as a question. Rewrite it as a goal that names the file that will exist when you are done. Hand the agent only the second version.

2. You do not build the loop

Claude Code, Codex, Cursor, GrokBot, Windsurf, Warp, Hermes. Different skins. Same loop. People say they built an agent. They tuned one.

Onboard it like a hire. A raw model is a capable stranger. Context is who you are. Skills are how you want the work done. Tools are how it touches the world. The model is only the Think step.

3. Observe · Think · Act

Look at state. Decide one next action. Do it. Repeat until the goal matches done. A real job can loop twenty or thirty times. Vague goals produce vague stops.

Step	What happens
Observe	Files, tools, last output, which skill applies
Think	The model picks one next action. This is the only place model brand matters.
Act	Write a file, call a tool or MCP, or launch a nested loop for one subgoal
Done	The artifact you named exists. If you cannot point at a path, you are still in chat.

4. Four levers

Lever	Loop step	What you change
Context	Observe	Markdown the agent should already know
Skills	Observe	SOPs for work you will ask twice
Model	Think	Which LLM reasons
Tools	Act	Reach into inbox, CRM, browser, files

A tuned mid-range model with your files beats a stock frontier model with none. Do not swap harnesses every week. Pull a lever on the one you already opened.

5. Context, pointers, memory

The session window is a workbench. The folder is the warehouse. Long threads rot. Early instructions fall out. New goal, new session. The north star reloads, so you lose nothing.

Only AGENTS.md or CLAUDE.md should auto-load. Everything else is pointed at and pulled when needed. Required sentence:

> *Everything you should know lives in /context and /skills. Load only what this goal needs. Assumptions are the enemy. If it is not in a file, ask.*

File	Job
AGENTS.md / CLAUDE.md	North star. Pointers only.
about_me.md	Who is speaking
business.md	What you sell and what done means
icp.md	Who you want. One wrong line poisons copy.
voice.md	Tone rules
memory.md	Dated lessons the agent appends. Humans strike. No secrets.

Vendor chat memory is a black box you cannot export. Owned markdown travels to any harness.

6. Skills are SOPs

Context is who you are. A skill is how the work gets done. Three parts: name, description with trigger phrases, body. The description is how the agent finds the book on the shelf. If you explain it twice, you found a missing skill.

Retrieval for a starter OS is search plus read. Do not stand up a vector database for twelve markdown files.

7. Tools, MCP, gates

Without tools the agent only talks. MCP is a translator into Gmail, Notion, HubSpot, a browser, a scraper marketplace. Translation is not permission. Read only first. Human gate before send, publish, spend, delete, or production write.

Act family	Default
Read files, search, draft	Allowed
Write CRM, send email, text, invoice	Human gate
New connector	Read only until earned
Secrets	Stay in .env. Never memory.md

8. Sub-agent vs more boxes

A sub-agent is a nested copy of the same loop for one subgoal when the parent window would rot. It is not a new employee. A graph is only worth it when roles need different permissions. Default: one loop, many skills.

Part II · Loop engineering

After single-shot prompting, leverage moves from the sentence you typed to the cycle you designed. You stop writing the output. You design the system that writes, checks, and iterates until a numeric stop is true.

Perceive → Reason → Act → Observe

Same family as Observe Think Act. Named this way when the job is research or production work with a score.

Stage	In an operator business
Perceive	Inbox, CRM row, PDF, last run, the skill that applies
Reason	Hypothesis or next step. Never treat self-score as measured truth.
Act	Extract, draft, classify, write a file, call an MCP
Observe	Did the artifact exist? Did the validator pass? Log the failure class.

Three nested timescales

Do not let a syntax fix block a week of strategy. Give each loop its own cap.

Loop	Timescale	Stops when
Micro	Seconds	The step runs, or three failed fixes
Inner	Minutes	One ticket, one lead row, one invoice is accepted or rejected
Outer	Hours to days	The week’s quota of accepted artifacts, each clearing the bar

State must be explicit. attempted, accepted, failed, why it failed, current artifact path, should_stop, stop_reason. No hidden side effects in a chat scrollback.

Five failure modes to design against

Failure	Defense
Confidence hallucination	Only measured checks count. The model’s “this looks good” does not.
Context overflow	Bound the live window. Long memory lives in files, not the thread.
Memory poisoning	Execution errors do not get written as “this strategy is bad.”
Runaway micro loop	Hard cap. Three code or extract retries, then skip.
Missing stop	Count plus quality plus diversity. A quota of junk is not done.

Worked onto GrokFlow lanes

Lane	Done looks like
Lead generation	A row with next action, source, and no invented email
Customer service	Ticket routed or resolved and logged in the system of record
Documents	Fields extracted, source cited, UNVERIFIED marked
App to app	Record written once, proof it landed
Invoicing	Invoice drafted from clean data. Send is a human gate.
Data hygiene	Duplicates flagged. Silent fills forbidden.

Part III · Harness engineering

The loop is what runs most often. The harness is the scaffolding the loop runs inside: model choice, context assembly, memory policy, tools, sandbox, what counts as a pass, who may send, what gets logged.

Loop engineering is the application layer. Harness engineering is the substrate. Do not conflate them. Do not spawn a new named bot when you needed a new skill or a new gate.

Treat the harness as an object

H = model configuration plus harness configuration. If you cannot version it, you cannot roll it back. Edits should be add, remove, replace, reorder on a named hook. Not a 4,000 word prompt patch no one can diff.

Nine dimensions you can actually touch

D	Dimension	Operator meaning
1	Model	Strong model for judgment. Cheap model for retries.
2	Context	What gets loaded before Reason
3	Memory	What is kept, decayed, never stored
4	Tools	Registry, schemas, MCP
5	Environment	Where code runs. Isolated from production money movement.
6	Evaluation	The check that is allowed to say pass
7	Control	Stop rules, permissions, human gates
8	Observability	Traces and session notes you can read
9	Training	Out of scope for a starter OS. Do not fine-tune your way out of missing files.

When the loop is fine and the harness is not

If the same class of job keeps failing, do not add another persona. Read the traces. Name the dimension. Change one thing. Re-run work you already passed so a “fix” does not wipe a working path. That is the operator version of a seesaw check.

Pathology	What you do
Reward hacking	A metric moved and something unmeasured broke. Read the side effects.
Forgetting	A new skill helped invoices and wrecked lead lists. Keep variants or roll back.
Under-exploration	You only ever tweak the prompt. Try a new tool, a new check, or a new split of context.

Advanced stacks add a digester that compresses traces, a planner that must propose one structural edit, an evolver that emits a manifest, a critic that hunts non-local damage, and a non-LLM gate that actually commits. You do not need that machine on Day 1. You need files, a stop rule, and honesty about what passed.

Part IV · The war-room harness

This section is the technical layer behind MyGrokFlow systems work. Source architecture: HarnessX (Chen et al., arXiv:2606.14249) plus loop engineering as used in a research war room. You do not need to train model weights. You do need a harness you can version.

Harness as an object

H = (M, C). M is the model and sampling. C is everything else, split into processors P attached to lifecycle hooks and slots S (tool registry, tracer). If C is not serializable you cannot diff it, roll it back, or evolve it. Edits are add, remove, replace, reorder at a hook. Not a 4,000-word prompt nobody can review.

Hook	What it may change
task_start	Session system prompt
before_reason	Context that hits hypothesis generation
after_reason	Raw hypothesis before Act
before_act	Code or tool input
after_act	Generated action before it runs
before_observe	Raw result before scoring
after_observe	Scored result before memory write
session_end	Handoff and digest

AEGIS: evolve the substrate, not the job

The loop mines work. AEGIS watches traces across many loops and edits the harness. One evolution round per N completed outer loops. LLM stages propose. Code decides.

Stage	Job	Defends against
Digester	Compress traces into failure clusters tagged D1–D9	Context overflow
Planner	Adaptation landscape plus one untried structural edit	Under-exploration
Evolver	Typed candidates, change manifest, smoke tests	Illegal patches
Critic	Non-local effects. One revision, then a ranked list	Reward hacking
Gate	Manifest, smoke, seesaw, no live write-paths	Forgetting and talk-into-prod

Seesaw: if the candidate regresses any previously passing scenario, reject. No LLM override. Variant isolation scopes that check per job family so a lead-gen fix does not have to wait on an invoicing variant.

What the paper measured

- Harness evolution: about +14.5% average success across reported model–benchmark pairs, up to +44% on a weaker baseline.
- Inverse scaling: harness quality helps most where the model cannot self-correct.
- Global harnesses stall on mixed work. Variants hold the gain.
- Co-evolving model weights is out of scope for a starter MyGrokFlow install. Stub D9. Do not fine-tune your way out of missing files.

Part V · War-room loop map

Loop engineering is the application layer. Perceive → Reason → Act → Observe until a numeric stop is true. Leverage moves from the prompt to the cycle.

Loop stage	Harness dimension
Perceive	D2 context assembly
Reason	D1 model + D2 context
Act	D4 tools + D7 control
Observe	D6 evaluation
Stop hook	D7 control
LoopState	D3 memory

Stop hook, non-negotiable

- Count of accepted artifacts is never enough by itself.
- Every accepted item clears the quality bar. Averages hide junk.
- Diversity check so you do not accept five near-duplicates.
- The agent cannot self-soften the hook. Only a human changes thresholds, in writing.

Models per seat, not one brain for all work

Seat	Model posture
Micro fix / extract	Fast cheap model. Hard cap of three retries.
Planner / Reason	Stronger reasoning model
Critic / gate advice	Strongest available. This is the last look before a harness edit.
Keys	Separate keys per seat if you need cost isolation. Never in memory.md.

Part VI · Principal-agent brief

Give this to Claude Code, Codex, Cursor, or GrokBot when you stand up a war room. Swap the domain. Do not skip the layering.

You are the principal agent. Design the system. Do not write product code until the phasing plan is approved.

Loop engineering is the application layer: many Perceive–Reason–Act–Observe cycles per harness generation. Harness engineering is the substrate: model, context, memory, tools, verifier, gates. AEGIS watches traces and evolves the harness. It does not replace the loop.

Folder skeleton

Path	Holds
harness/	Versioned H = (M, C)
loop-engine/	Perceive Reason Act Observe
aegis/	Digester Planner Evolver Critic
traces/ digests/ candidates/	Raw runs, compressed failure clusters, manifests
replay-buffer/ state/ handoff/	Trajectories, LoopState, dated facts
anchors/	Rules the agent may not soften

Phasing

Phase	Exit state
1	H is serializable. No loop yet.
2	One hypothesis reaches score through micro + inner loop.
3	Outer loop, Stop Hook, persisted LoopState.
4	Digester + Planner observe. No edits yet.
5	Evolver + Critic + deterministic gate close.
6	Variants per job family. Seesaw scoped per variant.

First output the principal must return

- Restate loop vs harness in their own words.
- Propose D6 metrics for this domain. Flag where a borrowed score does not transfer.
- Phasing plan with a demoable exit per phase.
- At most five open questions, ranked by what blocks the build.

Hard constraints you keep even if the domain changes

- Research output is artifacts a human reviews. No auto-promotion into production money movement.
- Sandbox stays isolated. No silent write-path to live systems.
- D9 training bridge stays stubbed until you explicitly open it.
- Stop Hook and seesaw are not negotiable by the loop.
- Domain facts that affect money or compliance are verified, not remembered.

Worked example domain in the source war room: research-only signal mining with a human gate before anything is considered for live use. Use that pattern. Do not copy a live desk into a public download.

Part VII · Install this week

Folder

project-root / AGENTS.md / context / skills / tools / memory.md

Open that folder in one harness for two weeks. Global files are how you work. Project files are this offer, this ICP, this CRM.

7 day drill

Day	Do this
1	Rewrite one task as a goal that names an output file
2	Write the loop in one line and run it once
3	Name the lever you are pulling
4	Fill about_me.md and business.md
5	New session. Append one lesson to memory.md
6	Write one skill you have explained twice
7	Finish the job. Stop only when the file exists

Order that actually ships

- One harness. Stay there.
- North star with the pointer sentence.
- One real goal.
- A skill when you correct the model twice.
- The smallest tool that removes a copy-paste.
- Gates on irreversible acts.
- Then a second harness with the same files.

Write yours. Then teach it.

A playbook only changes a company when someone fills the blanks and walks it into a meeting. Do this on paper or in your AGENTS.md. Then send the filled page to the person who still lives in chat.

1. The workflow you will take off your plate

Field	Write it here
Painful recurring job	_____
Who does it today	_____
File that exists when done	_____
System of record	CRM / inbox / books / other: _____
Human gate	send / spend / delete / none: _____
Lane	leads / CS / docs / invoice / hygiene

2. The sentence you will repeat

Chat is question to answer. An agent is goal to result. We are not buying another canvas. We are putting this job in a folder with a stop rule.

3. How you influence the room

- Do not demo a chatbot. Demo the Day 7 file.
- Ask what already waits on one person every week.
- Name the lever: context, skill, model, or tool. If they cannot name it, they are chatting.
- Offer one skill, not a 20-box org chart.
- Keep send, spend, and delete behind a human. That is how you get a yes.

Your north star draft

Copy this into AGENTS.md. Replace the brackets. This is how you take ownership of the playbook.

Line	Your version
Who we are	[company] helps [ICP] get [result].
What the agent may finish	[one workflow]. Output path: [file].
What it must ask	If it is not in /context, mark UNVERIFIED.
What it must never do alone	[send / pay / delete / deploy].
Where truth lives	[CRM / books / inbox]. Not the chat.

When this page is filled, the PDF is no longer ours. It is your operating system.

Self test

- Chat is ____ to ____. Agents are ____ to ____.
- Name Observe, Think, Act and what happens in each.
- Four levers and their loop step.
- Why a new session per goal?
- The sentence that belongs in every north star.
- Three parts of a skill.
- What MCP is, in one sentence.
- Why read-only is the default for a new tool.
- When a sub-agent is allowed.
- Loop vs harness, in one sentence each.
- Name one nested-loop cap you will enforce.
- What file did you produce on Day 7?

Pass: ten of twelve in your words, plus an artifact on disk.

Book the wiring session

MyGrokFlow installs what a free folder does not: MCP maps, quality gates, and automation that writes back to the systems you already paid for. Lead generation, customer service, documents, invoicing, data hygiene.

Bring the Day 7 file to mygrokflow.com. We turn that workflow into a system that runs without you.

MYGROKFLOW.COM · Request a diagnostic